Computación y Estructuras Discretas III

Andrés A. Aristizábal P.
aaaristizabal@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes

2026-1

Agenda del día

1. **Grammars and Languages**
   - Topic presentation

2. **Turing machines**
   - Topic presentation
   - Computing functions with Turing machines
   - Exercises

Agenda del día

1. **Grammars and Languages**
   - Topic presentation

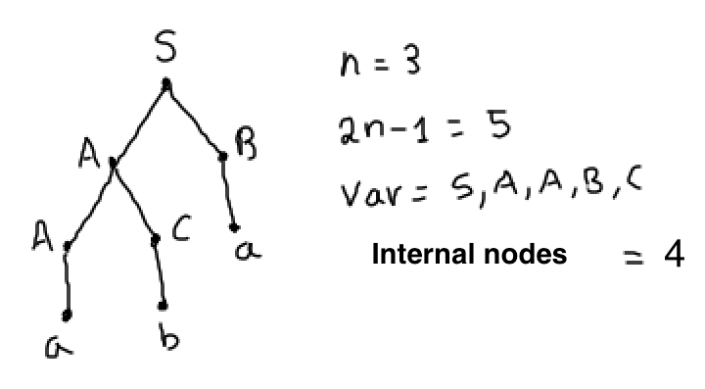
2. **Turing machines**
   - Topic presentation
   - Computing functions with Turing machines
   - Exercises

Grammars and Languages

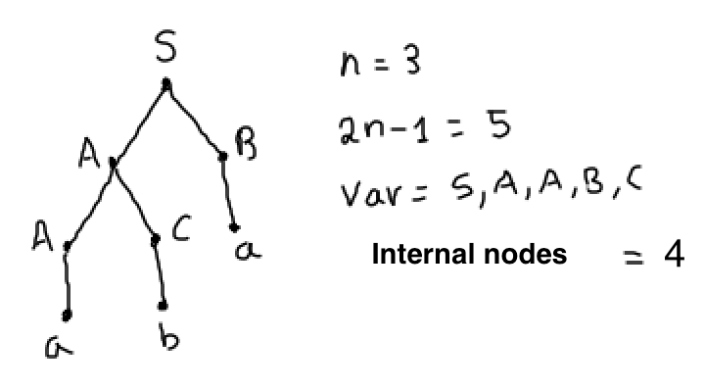
- A derivation tree for a string of length n has exactly $2n - 1$ nodes labeled with variables.

Grammars and Languages

- A derivation tree for a string of length n has exactly $2n - 1$ nodes labeled with variables.



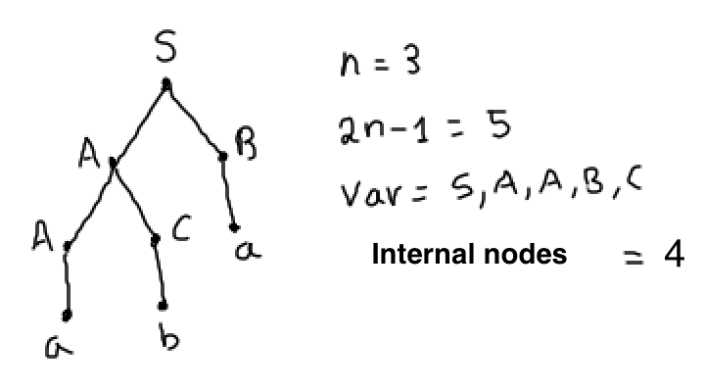
- A derivation tree for a string of length n has exactly $2n - 1$ nodes labeled with variables.



Since the root of the tree is S and S is not recursive, in a derivation tree for a string of length n, there are exactly $2n - 2$ interior nodes labeled with variables.

Grammars and Languages

- A derivation tree for a string of length n has exactly $2n - 1$ nodes labeled with variables.

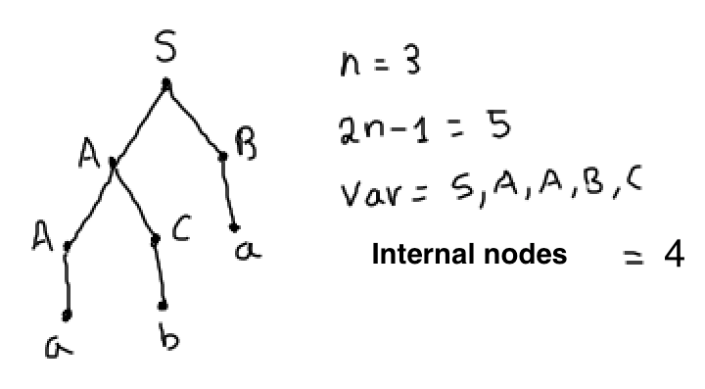


  Since the root of the tree is S and S is not recursive, in a derivation tree for a string of length n, there are exactly $2n - 2$ interior nodes labeled with variables.

- To determine whether a given string of length n is generated by G or not, all possible derivation trees with $2n - 2$ interior variables must be considered.

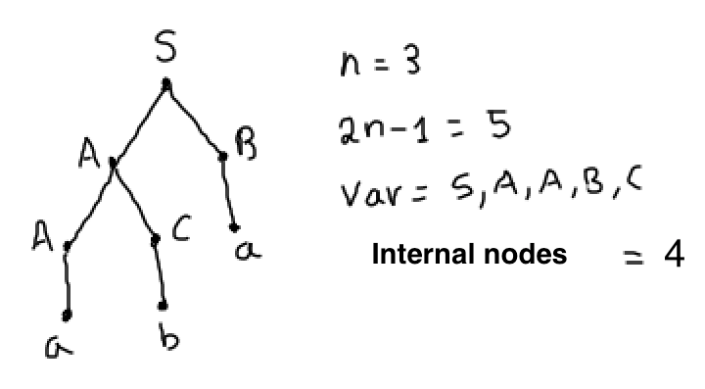
Grammars and Languages

- A derivation tree for a string of length n has exactly $2n - 1$ nodes labeled with variables.



  Since the root of the tree is S and S is not recursive, in a derivation tree for a string of length n, there are exactly $2n - 2$ interior nodes labeled with variables.
- To determine whether a given string of length n is generated by G or not, all possible derivation trees with $2n - 2$ interior variables must be considered.
- This algorithm is inefficient because if G has k variables, at least k^{2n-2} trees must be checked (not counting possibilities for the leaves).

- A derivation tree for a string of length n has exactly $2n - 1$ nodes labeled with variables.



  Since the root of the tree is S and S is not recursive, in a derivation tree for a string of length n, there are exactly $2n - 2$ interior nodes labeled with variables.
- To determine whether a given string of length n is generated by G or not, all possible derivation trees with $2n - 2$ interior variables must be considered.
- This algorithm is inefficient because if G has k variables, at least k^{2n-2} trees must be checked (not counting possibilities for the leaves).
- Therefore, the procedure has exponential complexity with respect to the input size.

Is there any efficient algorithm to solve this problem?

Is there any efficient algorithm to solve this problem?

- Yes, there is a very efficient algorithm to solve the membership problem (its complexity is polynomial).

Is there any efficient algorithm to solve this problem?

- Yes, there is a very efficient algorithm to solve the membership problem (its complexity is polynomial).
- It uses a technique called dynamic programming (a technique for filling tables progressively, reusing previously obtained information).

Is there any efficient algorithm to solve this problem?

- Yes, there is a very efficient algorithm to solve the membership problem (its complexity is polynomial).
- It uses a technique called dynamic programming (a technique for filling tables progressively, reusing previously obtained information).
- This algorithm is called the CYK algorithm (named after the researchers Cocke, Younger, and Kasami).

What is the CYK algorithm?

What is the CYK algorithm?

- This algorithm takes as input a CFG G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.

What is the CYK algorithm?

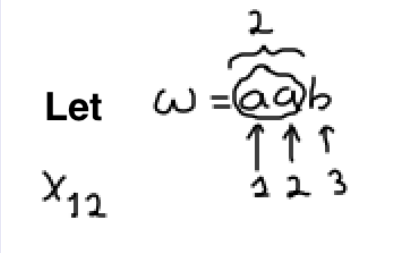
- This algorithm takes as input a CFG G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.
- It is applied by filling a table with n rows (one for each terminal of the input w) and n columns.

What is the CYK algorithm?

- This algorithm takes as input a CFG G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.
- It is applied by filling a table with n rows (one for each terminal of the input w) and n columns.
- X_{ij} is the set of variables from which the substring of w whose first symbol is at position i and whose length is j can be derived.

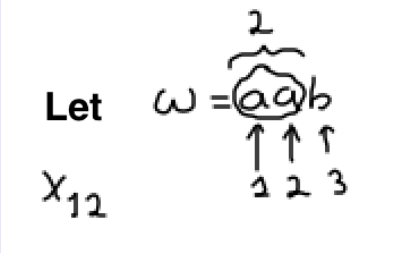
What is the CYK algorithm?

- This algorithm takes as input a CFG G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.
- It is applied by filling a table with n rows (one for each terminal of the input w) and n columns.
- X_{ij} is the set of variables from which the substring of w whose first symbol is at position i and whose length is j can be derived.



What is the CYK algorithm?

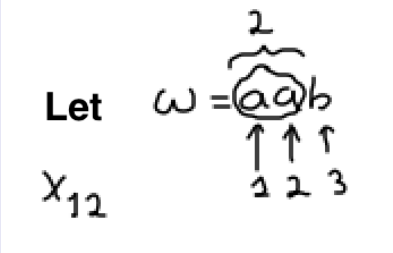
- This algorithm takes as input a CFG G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.
- It is applied by filling a table with n rows (one for each terminal of the input w) and n columns.
- X_{ij} is the set of variables from which the substring of w whose first symbol is at position i and whose length is j can be derived.



- That is, X_{ij} = set of variables A such that $A \overset{+}{\Longrightarrow} a_i a_{i+1} \cdots a_{i+j-1}$.

What is the CYK algorithm?

- This algorithm takes as input a CFG G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.
- It is applied by filling a table with n rows (one for each terminal of the input w) and n columns.
- X_{ij} is the set of variables from which the substring of w whose first symbol is at position i and whose length is j can be derived.



- That is, X_{ij} = set of variables A such that $A \overset{+}{\Longrightarrow} a_i a_{i+1} \cdots a_{i+j-1}$.
- By determining the sets X_{ij}, the possible ways to derive substrings of w that allow building a derivation of the complete string w are obtained.

- The table is filled column by column, from top to bottom; the first column ($j = 1$) corresponds to substrings of length 1, the second column ($j = 2$) corresponds to substrings of length 2, and so on. The last column ($j = n$) corresponds to the only substring of length n in w, which is the string w itself.

- The table is filled column by column, from top to bottom; the first column ($j = 1$) corresponds to substrings of length 1, the second column ($j = 2$) corresponds to substrings of length 2, and so on. The last column ($j = n$) corresponds to the only substring of length n in w, which is the string w itself.
- Variables that derive terminals at different positions in the string are found (first column).

- The table is filled column by column, from top to bottom; the first column ($j = 1$) corresponds to substrings of length 1, the second column ($j = 2$) corresponds to substrings of length 2, and so on. The last column ($j = n$) corresponds to the only substring of length n in w, which is the string w itself.
- Variables that derive terminals at different positions in the string are found (first column).
- Since j represents length, when $j = 1$, we find the variables that generate strings of length 1, and those are terminals.

- The table is filled column by column, from top to bottom; the first column ($j = 1$) corresponds to substrings of length 1, the second column ($j = 2$) corresponds to substrings of length 2, and so on. The last column ($j = n$) corresponds to the only substring of length n in w, which is the string w itself.
- Variables that derive terminals at different positions in the string are found (first column).
- Since j represents length, when $j = 1$, we find the variables that generate strings of length 1, and those are terminals.
- It follows that $w \in L(G)$ if and only if $S \in X_{1n}$.

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

$X_{11} = \{A\}$

$$w = \overbrace{a}^{1}\,a\,b$$

$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

$X_{11} = \{A\}$

$$w = \overbrace{a}^{1}\,a\,b$$

$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

$X_{21} = \{A\}$

$$w = a\,\overbrace{a}^{1}\,b$$

$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

$X_{11} = \{A\}$

$$w = \overbrace{a}^{1}\ a\ b$$
$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

$X_{21} = \{A\}$

$$w = a\ \overbrace{a}^{1}\ b$$
$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

$X_{31} = \{B\}$

$$w = a\ a\ \overbrace{b}^{1}$$
$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

$X_{11} = \{A\}$

$$w = \overbrace{a}^{1}\ a\ b$$

$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

$X_{12} = \{A\}$

$$w = \overbrace{a\ a}^{2}\ b$$

$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

AA

$X_{21} = \{A\}$

$$w = a\ \overbrace{a}^{1}\ b$$

$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

$X_{31} = \{B\}$

$$w = a\ a\ \overbrace{b}^{1}$$

$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

$X_{11} = \{A\}$

$$w = \overbrace{a}^{1}\ a\ b \qquad \begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

$X_{12} = \{A\}$

$$w = \overbrace{a\ a}^{2}\ b \qquad \begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

AA

$X_{21} = \{A\}$

$$w = a\ \overbrace{a}^{1}\ b \qquad \begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

$X_{22} = \{S\}$

$$w = a\ \overbrace{a\ b}^{2} \qquad \begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

AB

$X_{31} = \{B\}$

$$w = a\ a\ \overbrace{b}^{1} \qquad \begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

$X_{11} = \{A\}$

$$w = \overbrace{a}^{1}\,a\,b \qquad \begin{matrix} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{matrix}$$

$X_{12} = \{A\}$

$$w = \overbrace{a\,a}^{2}\,b \qquad \begin{matrix} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{matrix}$$

$X_{13} = \{S\}$

$$w = \overbrace{a\,a\,b}^{3} \qquad \begin{matrix} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{matrix}$$

$X_{21} = \{A\}$

$$w = a\,\overbrace{a}^{1}\,b \qquad \begin{matrix} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{matrix}$$

$X_{22} = \{S\}$

$$w = a\,\overbrace{a\,b}^{2} \qquad \begin{matrix} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{matrix}$$

$X_{31} = \{B\}$

$$w = a\,a\,\overbrace{b}^{1} \qquad \begin{matrix} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{matrix}$$

AA

AB

AS

AB

What is the pseudocode for the CYK algorithm?

What is the pseudocode for the CYK algorithm?

Algoritmo CYK

INPUT:
Gramámar G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.

INITIALIZE:
$j = 1$. For each i, $1 \leq i \leq n$,
$X_{ij} = X_{i1}$:= set of variables A such that $A \to a_i$

REPEAT:
$j := j + 1$. For each i, $1 \leq i \leq n - j + 1$,
X_{ij} := set of variables A such that $A \to BC$ es
a production of G, with $B \in X_{ik}$ y $C \in X_{i+k,j-k}$,
taking into account all k such that $1 \leq k \leq j - 1$.

UNTIL: $j = n$.

OUTPUT: $w \in L(G)$ if and only if $S \in X_{1n}$.

Example

Let G be the grammar

$$G : \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases}$$

Example

Let G be the grammar

$$G : \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases}$$

and the string $w = aab$. *Use the CYK algorithm to determine if* $w \in L(G)$ *or not.*

Solution

		$j=1$	$j=2$	$j=3$
a	$i=1$			
a	$i=2$			
b	$i=3$			

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases}$$

Solution

		$j=1$	$j=2$	$j=3$
a	$i=1$	$\{A\}$		
a	$i=2$	$\{A\}$		
b	$i=3$	$\{B\}$		

$X_{11} = \{A\}$
$X_{21} = \{A\}$
$X_{31} = \{B\}$

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases}$$

Solution

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases}$$

		$j=1$	$j=2$	$j=3$
a	$i=1$	$\{A\}$	$\{A\}$	
a	$i=2$	$\{A\}$	$\{S\}$	
b	$i=3$	$\{B\}$		

$X_{11} = \{A\}$

$X_{21} = \{A\}$

$X_{31} = \{B\}$

$X_{ij} \longrightarrow X_{ik} X_{i+k\, j-k}$

$j=2 \quad 1 \leq k \leq 1$

$i=1$

$X_{12} \to X_{11} X_{21}$

$X_{12} \to \{A\}\{A\} = \{AA\}$

$X_{12} = \{A\}$

$i=2$

$X_{22} \to X_{21} X_{31}$

$X_{22} \to \{A\}\{B\} = \{AB\}$

$X_{22} = \{S\}$

Solution

		$j=1$	$j=2$	$j=3$
a	$i=1$	$\{A\}$	$\{A\}$	$\{S\}$
a	$i=2$	$\{A\}$	$\{S\}$	
b	$i=3$	$\{B\}$		

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases}$$

$X_{11} = \{A\}$
$X_{21} = \{A\}$
$X_{31} = \{B\}$

$$X_{ij} \longrightarrow X_{ik} X_{i+k\, j-k}$$

$j=2 \quad 1 \leq k \leq 1$

$i=1$

$X_{12} \to X_{11} X_{21}$
$X_{12} \to \{A\}\{A\} = \{AA\}$
$X_{12} = \{A\}$

$i=2$

$X_{22} \to X_{21} X_{31}$
$X_{22} \to \{A\}\{B\} = \{AB\}$
$X_{22} = \{S\}$

$j=3 \quad 1 \leq k \leq 2$

$i=1$

$X_{13} \to X_{11} X_{22} \cup X_{12} X_{31}$
$X_{13} \to \{A\}\{S\} \cup \{A\}\{B\} = \{AS, AB\}$
$X_{13} = \{S\}$

$$S \in X_{13}$$

Example

Let G be the grammar

$$G : \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

Example

Let G be the grammar

$$G : \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

and the string $w = bbab$. Use the CYK algorithm to determine if $w \in L(G)$ or not.

Solution

		$j=1$	$j=2$	$j=3$	$j=4$
b	$i=1$				
b	$i=2$				
a	$i=3$				
b	$i=4$				

$$G: \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

Solution

		$j=1$	$j=2$	$j=3$	$j=4$
b	$i=1$	$\{A\}$			
b	$i=2$	$\{A\}$			
a	$i=3$	$\{B,C\}$			
b	$i=4$	$\{A\}$			

$X_{11} = \{A\}$
$X_{21} = \{A\}$
$X_{31} = \{B,C\}$
$X_{41} = \{A\}$

$$G: \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

Solution

		$j=1$	$j=2$	$j=3$	$j=4$
b	$i=1$	$\{A\}$	$\{\}$		
b	$i=2$	$\{A\}$	$\{B,S\}$		
a	$i=3$	$\{B,C\}$	$\{S,C\}$		
b	$i=4$	$\{A\}$			

$$G: \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

$X_{11} = \{A\}$
$X_{21} = \{A\}$
$X_{31} = \{B,C\}$
$X_{41} = \{A\}$

$$X_{ij} \longrightarrow X_{ik} X_{i+k\, j-k}$$

$j=2 \quad 1 \leq k \leq 1$

$i=1$

$X_{12} \to X_{11} X_{21}$
$X_{12} \to \{A\}\{A\} = \{AA\}$
$X_{12} = \{\}$

$i=2$

$X_{22} \to X_{21} X_{31}$
$X_{22} \to \{A\}\{B,C\} = \{AB, AC\}$
$X_{22} = \{B,S\}$

$i=3$

$X_{32} \to X_{31} X_{41}$
$X_{32} \to \{B,C\}\{A\} = \{BA, CA\}$
$X_{32} = \{S,C\}$

Solution

		$j=1$	$j=2$	$j=3$	$j=4$
b	$i=1$	$\{A\}$	$\{\}$	$\{B\}$	
b	$i=2$	$\{A\}$	$\{B,S\}$	$\{S,C\}$	
a	$i=3$	$\{B,C\}$	$\{S,C\}$		
b	$i=4$	$\{A\}$			

$$G: \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

$X_{11} = \{A\}$
$X_{21} = \{A\}$
$X_{31} = \{B,C\}$
$X_{41} = \{A\}$

$X_{ij} \longrightarrow X_{ik} X_{i+k\, j-k}$

$j=2 \quad 1 \le k \le 1$

$i=1$

$X_{12} \to X_{11} X_{21}$
$X_{12} \to \{A\}\{A\} = \{AA\}$
$X_{12} = \{\}$

$i=2$

$X_{22} \to X_{21} X_{31}$
$X_{22} \to \{A\}\{B,C\} = \{AB, AC\}$
$X_{22} = \{B,S\}$

$i=3$

$X_{32} \to X_{31} X_{41}$
$X_{32} \to \{B,C\}\{A\} = \{BA, CA\}$
$X_{32} = \{S,C\}$

$j=3 \quad 1 \le k \le 2$

$i=1$

$X_{13} \to X_{11} X_{22} \cup X_{12} X_{31}$
$X_{13} \to \{A\}\{B,S\} \cup \{\}\{B,C\} = \{AB, AS\}$
$X_{13} = \{B\}$

$i=2$

$X_{23} \to X_{21} X_{32} \cup X_{22} X_{41}$
$X_{23} \to \{A\}\{S,C\} \cup \{B,S\}\{A\} = \{AS, AC, BA, SA\}$
$X_{23} = \{S,C\}$

Solution

		$j=1$	$j=2$	$j=3$	$j=4$
b	$i=1$	$\{A\}$	$\{\}$	$\{B\}$	$\{S,C\}$
b	$i=2$	$\{A\}$	$\{B,S\}$	$\{S,C\}$	
a	$i=3$	$\{B,C\}$	$\{S,C\}$		
b	$i=4$	$\{A\}$			

$$G: \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

$X_{11} = \{A\}$
$X_{21} = \{A\}$
$X_{31} = \{B,C\}$
$X_{41} = \{A\}$

$X_{ij} \longrightarrow X_{ik} X_{i+k\, j-k}$

$j=2 \quad 1 \le k \le 1$

$i=1$

$X_{12} \to X_{11} X_{21}$
$X_{12} \to \{A\}\{A\} = \{AA\}$
$X_{12} = \{\}$

$i=2$

$X_{22} \to X_{21} X_{31}$
$X_{22} \to \{A\}\{B,C\} = \{AB, AC\}$
$X_{22} = \{B,S\}$

$i=3$

$X_{32} \to X_{31} X_{41}$
$X_{32} \to \{B,C\}\{A\} = \{BA, CA\}$
$X_{32} = \{S,C\}$

$j=3 \quad 1 \le k \le 2$

$i=1$

$X_{13} \to X_{11} X_{22} \cup X_{12} X_{31}$
$X_{13} \to \{A\}\{B,S\} \cup \{\}\{B,C\} = \{AB, AS\}$
$X_{13} = \{B\}$

$i=2$

$X_{23} \to X_{21} X_{32} \cup X_{22} X_{41}$
$X_{23} \to \{A\}\{S,C\} \cup \{B,S\}\{A\} = \{AS, AC, BA, SA\}$
$X_{23} = \{S,C\}$

$j=4 \quad 1 \le k \le 3$

$i=1$

$X_{14} \to X_{11} X_{23} \cup X_{12} X_{32} \cup X_{13} X_{41}$
$X_{14} \to \{A\}\{S,C\} \cup \{\}\{S,C\} \cup \{B\}\{A\} = \{AS, AC, BA\}$
$X_{14} = \{S,C\}$

$S \in X_{14}$

Agenda del día

1. Grammars and Languages
   - Topic presentation
2. **Turing machines**
   - Topic presentation
   - Computing functions with Turing machines
   - Exercises

Turing machines

What is a Turing machine?

Turing machines

What is a Turing machine?

A Turing Machine (TM), $M = (Q, q_0, F, \Sigma, \Gamma, b, \delta)$, consists of seven components:

Turing machines

What is a Turing machine?

A Turing Machine (TM), $M = (Q, q_0, F, \Sigma, \Gamma, b, \delta)$, consists of seven components:

1. Q is the (finite) set of internal states of the control unit.

Turing machines

What is a Turing machine?

A Turing Machine (TM), $M = (Q, q_0, F, \Sigma, \Gamma, b, \delta)$, consists of seven components:

1. Q is the (finite) set of internal states of the control unit.
2. $q_0 \in Q$ is the initial state.

Turing machines

What is a Turing machine?

A Turing Machine (TM), $M = (Q, q_0, F, \Sigma, \Gamma, b, \delta)$, consists of seven components:

1. Q is the (finite) set of internal states of the control unit.
2. $q_0 \in Q$ is the initial state.
3. F is the set of final or accepting states, $\emptyset \neq F \subseteq Q$.

Turing machines

What is a Turing machine?

A Turing Machine (TM), $M = (Q, q_0, F, \Sigma, \Gamma, b, \delta)$, consists of seven components:

1. Q is the (finite) set of internal states of the control unit.
2. $q_0 \in Q$ is the initial state.
3. F is the set of final or accepting states, $\emptyset \neq F \subseteq Q$.
4. Σ is the input alphabet.

Turing machines

What is a Turing machine?

A Turing Machine (TM), $M = (Q, q_0, F, \Sigma, \Gamma, b, \delta)$, consists of seven components:

1. Q is the (finite) set of internal states of the control unit.
2. $q_0 \in Q$ is the initial state.
3. F is the set of final or accepting states, $\emptyset \neq F \subseteq Q$.
4. Σ is the input alphabet.
5. Γ is the tape alphabet, which includes Σ, i.e., $\Sigma \subseteq \Gamma$.

Turing machines

What is a Turing machine?

A Turing Machine (TM), $M = (Q, q_0, F, \Sigma, \Gamma, b, \delta)$, consists of seven components:

1. Q is the (finite) set of internal states of the control unit.
2. $q_0 \in Q$ is the initial state.
3. F is the set of final or accepting states, $\emptyset \neq F \subseteq Q$.
4. Σ is the input alphabet.
5. Γ is the tape alphabet, which includes Σ, i.e., $\Sigma \subseteq \Gamma$.
6. $b \in \Gamma$ is the "blank" symbol (the symbol b cannot be part of the input alphabet Σ).

Turing machines

What is a Turing machine?

A Turing Machine (TM), $M = (Q, q_0, F, \Sigma, \Gamma, b, \delta)$, consists of seven components:

1. Q is the (finite) set of internal states of the control unit.
2. $q_0 \in Q$ is the initial state.
3. F is the set of final or accepting states, $\emptyset \neq F \subseteq Q$.
4. Σ is the input alphabet.
5. Γ is the tape alphabet, which includes Σ, i.e., $\Sigma \subseteq \Gamma$.
6. $b \in \Gamma$ is the "blank" symbol (the symbol b cannot be part of the input alphabet Σ).
7. δ is the transition function of the machine:

Turing machines

What is a Turing machine?

A Turing Machine (TM), $M = (Q, q_0, F, \Sigma, \Gamma, b, \delta)$, consists of seven components:

1. Q is the (finite) set of internal states of the control unit.
2. $q_0 \in Q$ is the initial state.
3. F is the set of final or accepting states, $\emptyset \neq F \subseteq Q$.
4. Σ is the input alphabet.
5. Γ is the tape alphabet, which includes Σ, i.e., $\Sigma \subseteq \Gamma$.
6. $b \in \Gamma$ is the "blank" symbol (the symbol b cannot be part of the input alphabet Σ).
7. δ is the transition function of the machine:

$$\delta : Q \times \Gamma \longrightarrow Q \times \Gamma \times \{\leftarrow, \rightarrow\}$$

Turing machines

What is a Turing machine?

A Turing Machine (TM), $M = (Q, q_0, F, \Sigma, \Gamma, b, \delta)$, consists of seven components:

1. Q is the (finite) set of internal states of the control unit.
2. $q_0 \in Q$ is the initial state.
3. F is the set of final or accepting states, $\emptyset \neq F \subseteq Q$.
4. Σ is the input alphabet.
5. Γ is the tape alphabet, which includes Σ, i.e., $\Sigma \subseteq \Gamma$.
6. $b \in \Gamma$ is the "blank" symbol (the symbol b cannot be part of the input alphabet Σ).
7. δ is the transition function of the machine:

$$\delta : Q \times \Gamma \longrightarrow Q \times \Gamma \times \{\leftarrow, \rightarrow\}$$

   - δ is a partial function, meaning it may not be defined for some elements of the domain.

Turing machines

What is a Turing machine?

A Turing Machine (TM), $M = (Q, q_0, F, \Sigma, \Gamma, b, \delta)$, consists of seven components:

1. Q is the (finite) set of internal states of the control unit.
2. $q_0 \in Q$ is the initial state.
3. F is the set of final or accepting states, $\emptyset \neq F \subseteq Q$.
4. Σ is the input alphabet.
5. Γ is the tape alphabet, which includes Σ, i.e., $\Sigma \subseteq \Gamma$.
6. $b \in \Gamma$ is the "blank" symbol (the symbol b cannot be part of the input alphabet Σ).
7. δ is the transition function of the machine:

$$\delta : Q \times \Gamma \longrightarrow Q \times \Gamma \times \{\leftarrow, \rightarrow\}$$

- δ is a partial function, meaning it may not be defined for some elements of the domain.
- The arrow $\leftarrow$ denotes a move to the left, while $\rightarrow$ denotes a move to the right.

Turing machines

What is a Turing machine?

A Turing Machine (TM), $M = (Q, q_0, F, \Sigma, \Gamma, b, \delta)$, consists of seven components:

1. Q is the (finite) set of internal states of the control unit.
2. $q_0 \in Q$ is the initial state.
3. F is the set of final or accepting states, $\emptyset \neq F \subseteq Q$.
4. Σ is the input alphabet.
5. Γ is the tape alphabet, which includes Σ, i.e., $\Sigma \subseteq \Gamma$.
6. $b \in \Gamma$ is the "blank" symbol (the symbol b cannot be part of the input alphabet Σ).
7. δ is the transition function of the machine:

$$\delta : Q \times \Gamma \longrightarrow Q \times \Gamma \times \{\leftarrow, \rightarrow\}$$

- δ is a partial function, meaning it may not be defined for some elements of the domain.
- The arrow $\leftarrow$ denotes a move to the left, while $\rightarrow$ denotes a move to the right.
- The transition $\delta(q, a) = (p, b, D)$ means: while in state q, scanning symbol a, the control unit erases a, writes b and moves to state p, either to the left (if the move D is $\leftarrow$) or to the right (if D is $\rightarrow$).

Turing machines

How does a Turing machine work?

Turing machines

How does a Turing machine work?

- A Turing Machine M processes input strings $w \in \Sigma^*$ placed on an infinite tape in both directions.
- To process an input string w, the control unit of M is in the initial state q_0 scanning the first symbol of w.
- The remaining cells or squares of the tape contain the blank symbol $\flat$.

Turing machines

How does a Turing machine work?

- A Turing Machine M processes input strings $w \in \Sigma^*$ placed on an infinite tape in both directions.
- To process an input string w, the control unit of M is in the initial state q_0 scanning the first symbol of w.
- The remaining cells or squares of the tape contain the blank symbol $\dot{b}$.

What is an instantaneous configuration?

Turing machines

How does a Turing machine work?

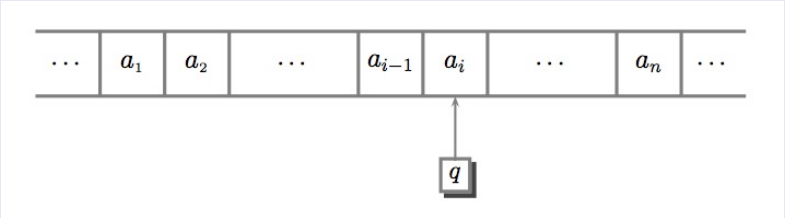
- A Turing Machine M processes input strings $w \in \Sigma^*$ placed on an infinite tape in both directions.
- To process an input string w, the control unit of M is in the initial state q_0 scanning the first symbol of w.
- The remaining cells or squares of the tape contain the blank symbol $\flat$.

What is an instantaneous configuration?

A description or instantaneous configuration is an expression of the form:

$$a_1 a_2 \cdots a_{i-1} q a_i \cdots a_n$$

where the symbols $a_1, \ldots, a_n$ belong to the tape alphabet Γ y $q \in Q$. This expression represents the current state of the computation:



Turing machines

What does the instantaneous configuration indicate?

Turing machines

What does the instantaneous configuration indicate?

The instantaneous description $a_1 a_2 \cdots a_{i-1} q a_i \cdots a_n$ indicates that the control unit of M is in state q scanning symbol a_i. It is assumed that all cells to the left of a_1 and to the right of a_n, contain the blank symbol $\bar{b}$.

Example

Examples of instantaneous descriptions are:

$$aabq_2baaa$$
$$q_5ababba$$
$$ab\bar{b}\bar{b}baabq_0bba$$

Turing machines

What does the instantaneous configuration indicate?

The instantaneous description $a_1 a_2 \cdots a_{i-1} q a_i \cdots a_n$ indicates that the control unit of M is in state q scanning symbol a_i. It is assumed that all cells to the left of a_1 and to the right of a_n, contain the blank symbol $\bar{b}$.

Example

Examples of instantaneous descriptions are:

$$aabq_2baaa$$
$$q_5ababba$$
$$ab\bar{b}\bar{b}baabq_0bba$$

What is the initial instantaneous configuration?

Turing machines

What does the instantaneous configuration indicate?

The instantaneous description $a_1 a_2 \cdots a_{i-1} q a_i \cdots a_n$ indicates that the control unit of M is in state q scanning symbol a_i. It is assumed that all cells to the left of a_1 and to the right of a_n, contain the blank symbol $\bar{b}$.

Example

Examples of instantaneous descriptions are:

$$aabq_2baaa$$
$$q_5ababba$$
$$ab\bar{b}\bar{b}baabq_0bba$$

What is the initial instantaneous configuration?

The initial instantaneous configuration, or simply the initial configuration, is $q_0 w$, where w is the input string. This is placed anywhere on the input tape.

Turing machines

What is a computational step?

Turing machines

What is a computational step?

The transition from one instantaneous description to another, through a transition defined by δ, is called a computational step and is denoted by:

$$u_1 q u_2 \vdash v_1 p v_2.$$

Here $u_1, u_2, v_1, v_2 \in \Gamma^*$ and $p, q \in Q$.

Turing machines

What is a computational step?

The transition from one instantaneous description to another, through a transition defined by δ, is called a computational step and is denoted by:

$$u_1 q u_2 \vdash v_1 p v_2.$$

Here $u_1, u_2, v_1, v_2 \in \Gamma^*$ and $p, q \in Q$.

Example

A concrete example is:

$$abbaq_2ba \vdash abbq_1aca$$

in which the machine used the transition $\delta(q_2, b) = (q_1, c, \leftarrow)$.

Turing machines

What is a computational step?

The transition from one instantaneous description to another, through a transition defined by δ, is called a computational step and is denoted by:

$$u_1 q u_2 \vdash v_1 p v_2.$$

Here $u_1, u_2, v_1, v_2 \in \Gamma^*$ and $p, q \in Q$.

Example

A concrete example is:

$$abbaq_2ba \vdash abbq_1aca$$

in which the machine used the transition $\delta(q_2, b) = (q_1, c, \leftarrow)$.

What does the notation $u_1 q u_2 \overset{*}{\vdash} v_1 p v_2$ indicate?

Turing machines

What is a computational step?

The transition from one instantaneous description to another, through a transition defined by δ, is called a computational step and is denoted by:

$$u_1 q u_2 \vdash v_1 p v_2.$$

Here $u_1, u_2, v_1, v_2 \in \Gamma^*$ and $p, q \in Q$.

Example

A concrete example is:

$$abbaq_2ba \vdash abbq_1aca$$

in which the machine used the transition $\delta(q_2, b) = (q_1, c, \leftarrow)$.

What does the notation $u_1 q u_2 \overset{*}{\vdash} v_1 p v_2$ indicate?

It means that M can transition from the instantaneous description $u_1 q u_2$ to the instantaneous description $v_1 p v_2$ in zero or more computational steps.

Turing machines

What are special computations?

What are special computations?

During the computation or processing of an input string, there are two special situations that can occur:

1. The computation terminates because at some point there is no defined transition.
2. The computation does not terminate; this is what is called an "infinite loop" or an "infinite cycle". This situation is represented by the notation

$$u_1 q u_2 \overset{*}{\vdash} \infty$$

which indicates that the computation starting from the instantaneous description $u_1 q u_2$ never stops.

Turing machines

Are transitions with the symbol $\not{b}$, $\delta(q, \not{b}) = (p, s, D)$, the same as the λ transitions used in automata?

Turing machines

Are transitions with the symbol $\not{b}$, $\delta(q, \not{b}) = (p, s, D)$, the same as the λ transitions used in automata?

- They are not the same.
- A λ transition en un autómata transition in an automaton occurs regardless of the symbol read, and the control unit remains stationary, whereas a $\delta(q, \not{b}) = (p, s, D)$ transition in a TM requires the blank symbol $\not{b}$ to be written on the scanned cell.
- The control unit overwrites the blank and performs a shift.

Turing machines

What is the language accepted by a Turing machine?

Turing machines

What is the language accepted by a Turing machine?

- An input string w is accepted by a Turing machine M if the computation that starts in the initial configuration $q_0 w$ ends in an instantaneous configuration $w_1 p w_2$, where p is an accepting state, and in which M halts completely.
- The language $L(M)$ accepted by a Turing machine M is then defined as

$$L(M) := \{w \in \Sigma^* \mid q_0 w \overset{*}{\vdash} w_1 p w_2, p \in F, w_1, w_2 \in \Gamma^*, M \text{ halts in the configuration } w_1 p w_2\}.$$

Turing machines

What is the language accepted by a Turing machine?

- An input string w is accepted by a Turing machine M if the computation that starts in the initial configuration $q_0 w$ ends in an instantaneous configuration $w_1 p w_2$, where p is an accepting state, and in which M halts completely.
- The language $L(M)$ accepted by a Turing machine M is then defined as

$$L(M) := \{w \in \Sigma^* \mid q_0 w \overset{*}{\vdash} w_1 p w_2, p \in F, w_1, w_2 \in \Gamma^*, M \text{ halts in the configuration } w_1 p w_2\}.$$

What difference exists between the notion of acceptance for Turing machines and for automata?

Turing machines

What is the language accepted by a Turing machine?

- An input string w is accepted by a Turing machine M if the computation that starts in the initial configuration $q_0 w$ ends in an instantaneous configuration $w_1 p w_2$, where p is an accepting state, and in which M halts completely.
- The language $L(M)$ accepted by a Turing machine M is then defined as

$$L(M) := \{w \in \Sigma^* \mid q_0 w \overset{*}{\vdash} w_1 p w_2, p \in F, w_1, w_2 \in \Gamma^*, M \text{ halts in the configuration } w_1 p w_2\}.$$

What difference exists between the notion of acceptance for Turing machines and for automata?

- The notion of acceptance for Turing machines is more flexible than for automata.
- An input string does not have to be read entirely to be accepted.
- It is only required that the machine halts completely, at a certain point, in an accepting state.

Turing machines

When does the control unit stop in a Turing machine?

Turing machines

When does the control unit stop in a Turing machine?

- In the standard model, the control unit always stops upon entering a in an accepting state.
- That is, transitions $\delta(q, a)$ are not allowed when $q \in F$.

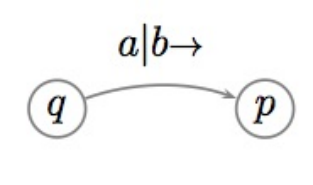
Turing machines

How are transition diagrams for Turing machines?

Turing machines

How are transition diagrams for Turing machines?

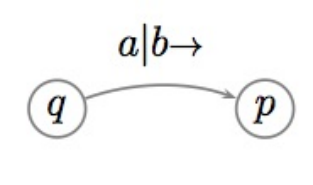
The transition function δ of a Turing machine can be presented as a labeled digraph. Thus, the transition $\delta(q, a) = (p, b, \rightarrow)$ s represented by



Turing machines

How are transition diagrams for Turing machines?

The transition function δ of a Turing machine can be presented as a labeled digraph. Thus, the transition $\delta(q, a) = (p, b, \rightarrow)$ s represented by

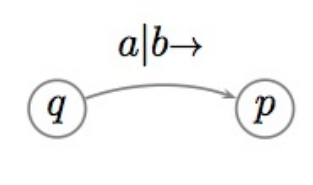


What is the stationary control unit?

Turing machines

How are transition diagrams for Turing machines?

The transition function δ of a Turing machine can be presented as a labeled digraph. Thus, the transition $\delta(q, a) = (p, b, \rightarrow)$ s represented by



What is the stationary control unit?

- To simplify the description of the standard model, it has been required that the control unit moves left or right on each transition.
- However, it is possible to allow the control unit to stay stationary during a certain computational step, meaning it does not move at all.

Turing machines

- This type of transition is written in the form:

$$\delta(q, a) = (p, b, -)$$

  where $a, b \in \Gamma$; $p, q \in Q$ and $-$ means that the control unit does not move.
  - Such a transition can be simulated by moving right followed by a return to the left.
  - Thus, to simulate the transition $\delta(q, a) = (p, b, -)$ we use a new auxiliary state q' and the transitions:

$$\delta(q, a) = (q', b, \rightarrow),$$
$$\delta(q', s) = (p, s, \leftarrow), \text{ for every symbol } s \in \Gamma.$$

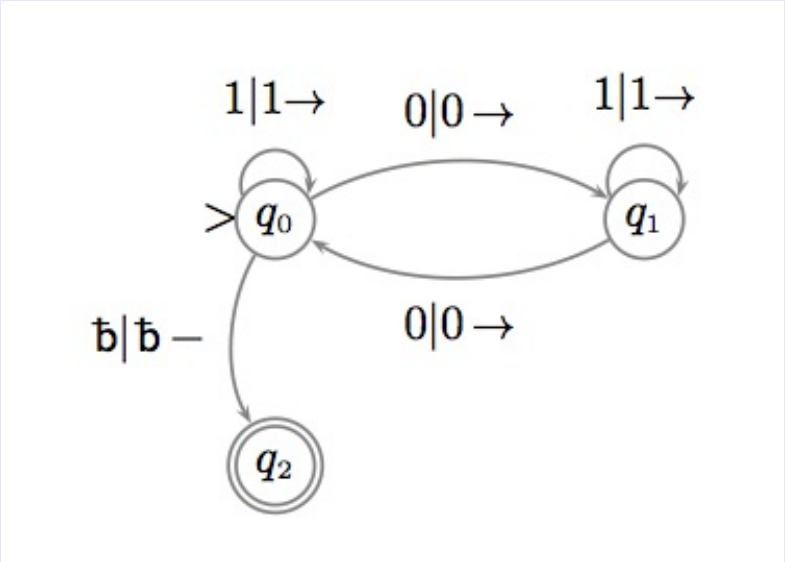
  - Stationary movement can also be simulated with a move to the left followed by a return to the right.
- It is concluded that Turing machines with stationary transitions, $\delta(q, a) = (p, b, -)$, in addition to normal transitions, accept the same languages as standard Turing machines.
- It is worth noting that stationary transitions resemble λ transitions in automata, except for the fact that Turing machines have the ability to overwrite scanned symbols.

Example

The following Turing machine accepts the language of strings with an even number of zeros, over the alphabet $0, 1$.

Example

The following Turing machine accepts the language of strings with an even number of zeros, over the alphabet $0, 1$.



Example

In this example, we built a Turing machine M that accepts the language $L = \{a^i b^i c^i \mid i \geq 0\}$ *and halts when processing all inputs. Therefore, L eis a recursive language although it is not context-free, meaning, L cannot be accepted by any pushdown automaton.*

Example

In this example, we built a Turing machine M that accepts the language $L = \{a^i b^i c^i \mid i \geq 0\}$ *and halts when processing all inputs. Therefore, L eis a recursive language although it is not context-free, meaning, L cannot be accepted by any pushdown automaton.*

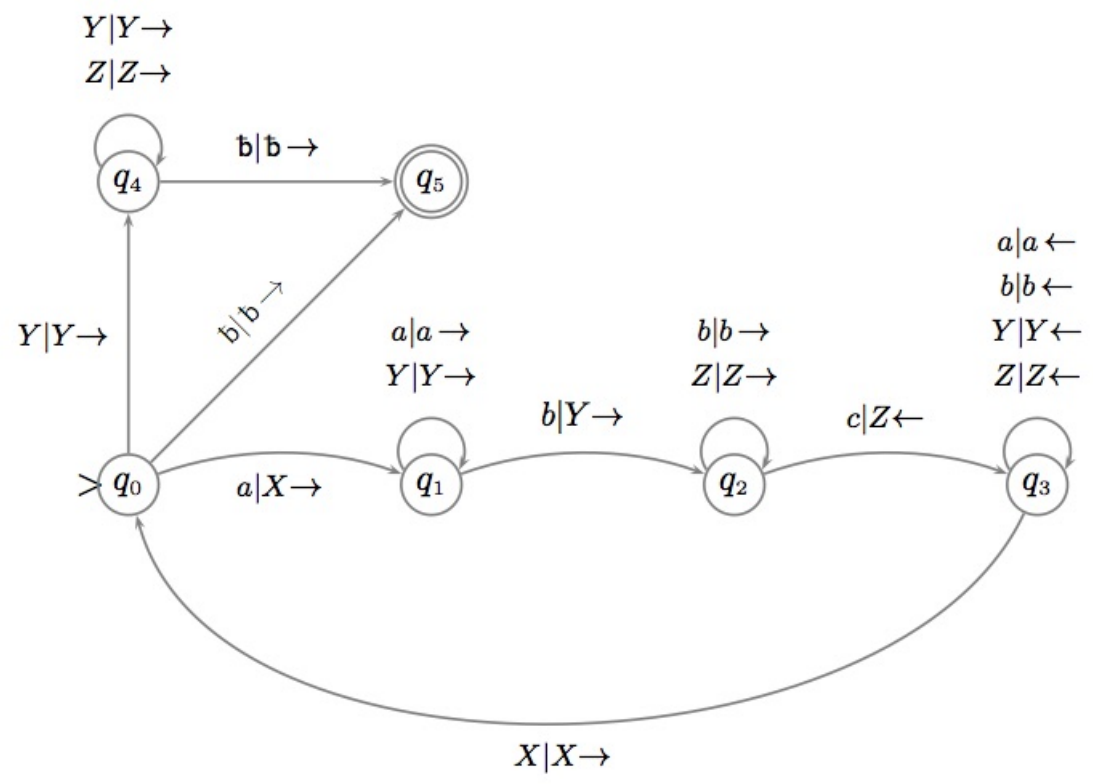
Solution

Let M be the Turing machine with parameters

$$\begin{aligned}
&\Sigma = \{a, b, c\},\\
&\Gamma = \{a, b, c, X, Y, Z, \dot{b}\},\\
&Q = \{q_0, q_1, q_2, q_3, q_4, q_5\},\\
&F = \{q_5\},
\end{aligned}$$

and whose transition function is represented by the following diagram:

Solution



Solution

The idea used for the design of this Turing machine can be described as follows:

Solution

The idea used for the design of this Turing machine can be described as follows:

- *The control unit changes the first a to X and moves to the right until it finds the first b, which is overwritten by a Y.*
- *Then it moves to the right until it finds the first c, which is changed to Z.*
- *The control then moves back to the left in search of the first X it encounters on its way; this return is done in state q_3.*
- *The machine then moves to the right, until the first a left on the tape, and the whole process is repeated.*
- *If the input string has the required form, all a's will be replaced by Xs, b's by Ys anc c's by Zs.*
- *Once the transformation is complete, the control moves to the right, in state q_4, until it finds the first cell marked with the blank symbol $\bar{b}$.*
- *The Turing machine is designed in such a way that if the input string does not have the required form, the processing will end in a state other than the accepting state q_5.*

Solution

Next, we process the input string $w = aabbcc \in L$.

$$
\begin{aligned}
q_0 aabbcc &\vdash Xq_1 abbcc \vdash Xaq_1 bbcc \vdash XaYq_2 bcc \vdash XaYbq_2 cc \vdash XaYq_3 bZc \\
&\overset{*}{\vdash} q_3 XaYbZc \vdash Xq_0 aYbZc \vdash XXq_1 YbZc \vdash XXYq_1 bZc \\
&\vdash XXYYq_2 Zc \vdash XXYZq_2 c \vdash XXYq_3 ZZ \overset{*}{\vdash} Xq_3 XYZZ \\
&\vdash XXq_0 YYZZ \vdash XXYq_4 YZZ \overset{*}{\vdash} XXYYZZq_4 \text{ƀ} \\
&\vdash XXYYZZ\text{ƀ}q_5 \text{ƀ}.
\end{aligned}
$$

The input string $w = aaabbcc$, *which is not in* L, *is processed as follows:*

$$
\begin{aligned}
q_0 aaabbcc &\vdash Xq_1 aabbcc \vdash Xaaq_1 bbcc \vdash XaaYq_2 bcc \vdash XaaYbq_2 cc \\
&\vdash XaaYbq_3 bZc \overset{*}{\vdash} q_3 XaaYbZc \vdash Xq_0 aaYbZc \vdash XXaq_1 YbZc \\
&\vdash XXaYq_1 bZc \vdash XXaYYq_2 Zc \vdash XXaYZq_2 c \vdash XXaYq_3 ZZ \\
&\overset{*}{\vdash} Xq_3 XaYZZ \vdash XXq_0 aYYZZ \vdash XXXq_1 YYZZ\text{ƀ} \\
&\overset{*}{\vdash} XXXYYq_1 ZZ \text{ (Cómputo abortado).}
\end{aligned}
$$

Example

In this example, we built a Turing machine M that accepts the language $L = \{a^i b^i c^i \mid i \geq 0\}$ *and halts when processing all inputs. Therefore, L eis a recursive language although it is not context-free, meaning, L cannot be accepted by any pushdown automaton.*

Example

In this example, we built a Turing machine M that accepts the language $L = \{a^i b^i c^i \mid i \geq 0\}$ *and halts when processing all inputs. Therefore, L eis a recursive language although it is not context-free, meaning, L cannot be accepted by any pushdown automaton.*

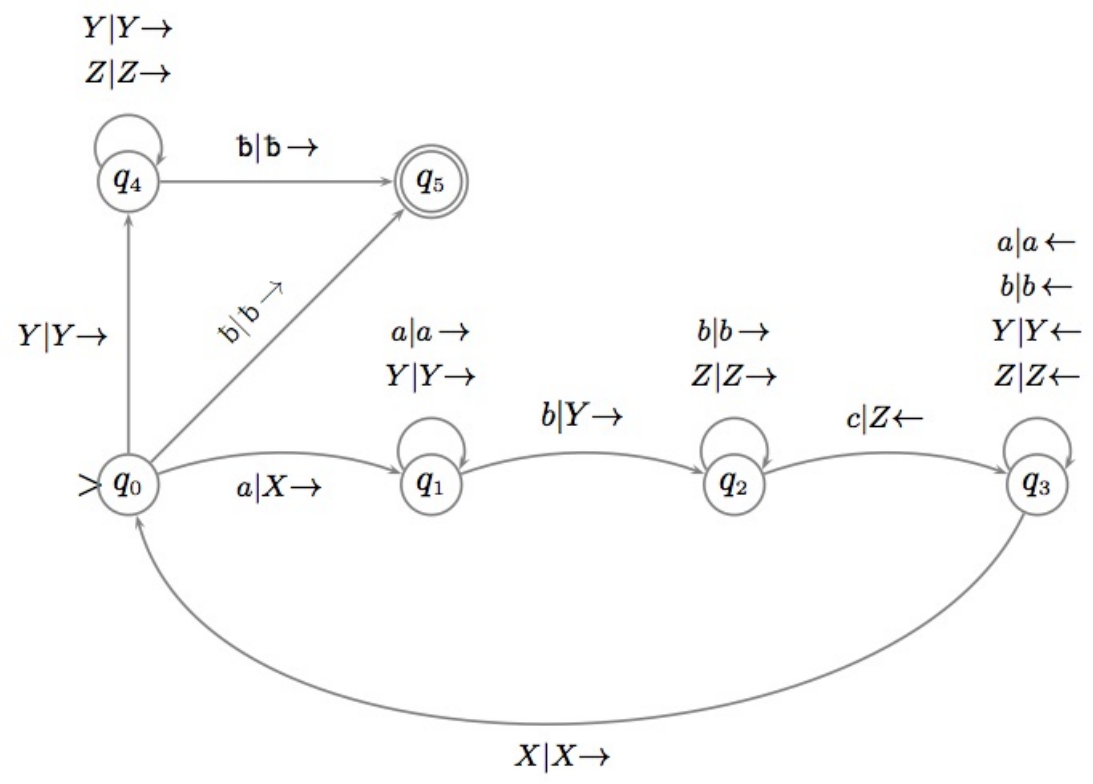
Solution

Let M be the Turing machine with parameters

$$\begin{aligned}
&\Sigma = \{a, b, c\},\\
&\Gamma = \{a, b, c, X, Y, Z, \dot{b}\},\\
&Q = \{q_0, q_1, q_2, q_3, q_4, q_5\},\\
&F = \{q_5\},
\end{aligned}$$

and whose transition function is represented by the following diagram:

Solution



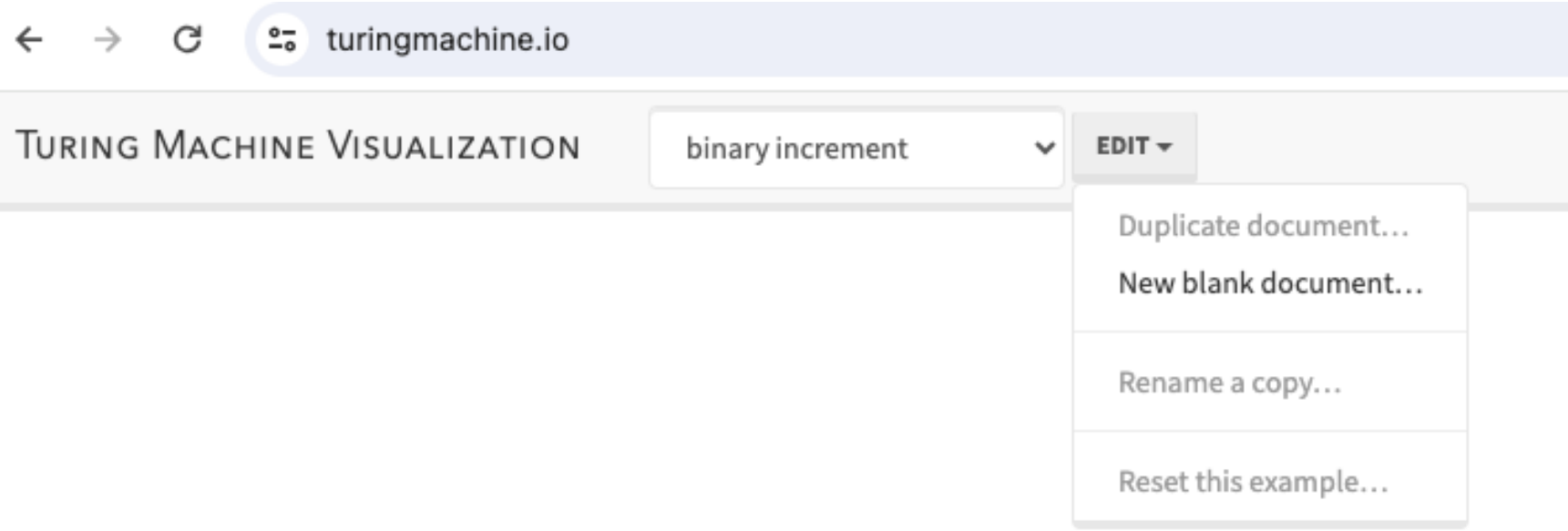
Turing machines

How can we simulate the previous Turing machine?

Turing machines

How can we simulate the previous Turing machine?

- We go to turingmachine.io
- Go to edit and new blank document



Turing machines

- We give it a name (e.g. example)
- Start coding our Turing machine

Turing machines

- The input string is where we will write the string used to simulate the TM with.
- blank represents the blank symbol
- start state is the initial state
- table represents que transition function of your TM

LOAD MACHINE

REVERT TO DIAGRAM

```
1  input: ''
2  blank: ' '
3  start state: start
4  table:
5    start:
6
7
```

Turing machines

- $\delta(q_0, a) = (q_1, b, \rightarrow)$
- Is written as:

```
input: ''
blank: ' '
start state: q0
table:
  q0:
    'a': {write: 'a', R: q1}
```

Turing machines

- $\delta(q_0, a) = (q_1, a, \leftarrow)$
- Is written as:

```
input: ''
blank: ' '
start state: q0
table:
  q0:
    'a': {write: 'a', L: q1}
```

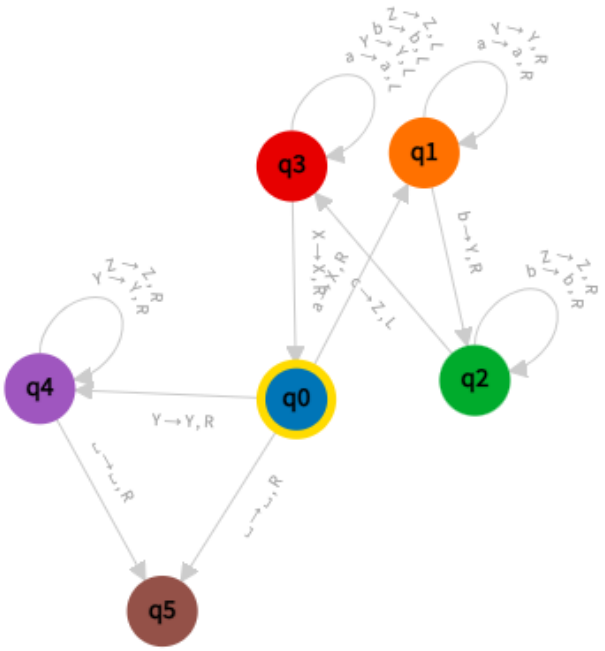
Turing machines

This would be the code for our example

```
input: 'aaabbbccc'
blank: ' '
start state: q0
table:
  q0:
    'a': {write: 'X', R: q1}
    'Y': {write: 'Y', R: q4}
    ' ': {write: ' ', R: q5}
  q1:
    'a': {write: 'a', R: q1}
    'Y': {write: 'Y', R: q1}
    'b': {write: 'Y', R: q2}
  q2:
    'b': {write: 'b', R: q2}
    'Z': {write: 'Z', R: q2}
    'c': {write: 'Z', L: q3}
  q3:
    'a': {write: 'a', L: q3}
    'Y': {write: 'Y', L: q3}
    'b': {write: 'b', L: q3}
    'Z': {write: 'Z', L: q3}
    'X': {write: 'X', R: q0}
  q4:
    'Y': {write: 'Y', R: q4}
    'Z': {write: 'Z', R: q4}
    ' ': {write: ' ', R: q5}
  q5:
```

Turing machines

And we can simulate it over the input string pressing run



Agenda del día

1. Grammars and Languages
   - Topic presentation
2. Turing machines
   - Topic presentation
   - Computing functions with Turing machines
   - Exercises

Turing Machines

What else can an MT be used for?

Turing Machines

What else can an MT be used for?

Since Turing machines have the ability to transform input strings, erase or overwrite symbols, they can also be used as mechanisms to compute functions.

Turing Machines

What else can an MT be used for?

Since Turing machines have the ability to transform input strings, erase or overwrite symbols, they can also be used as mechanisms to compute functions.

How can we formally define a Turing machine as a function calculator?

Turing Machines

What else can an MT be used for?

Since Turing machines have the ability to transform input strings, erase or overwrite symbols, they can also be used as mechanisms to compute functions.

How can we formally define a Turing machine as a function calculator?

Definición

A Turing machine $M = (Q, q_0, q_f, \Sigma, \Gamma, b, \delta)$ *computes a function* $h : \Sigma^* \to \Gamma^*$ *if for every input* $w \in \Sigma^*$ *we have*

Turing Machines

What else can an MT be used for?

Since Turing machines have the ability to transform input strings, erase or overwrite symbols, they can also be used as mechanisms to compute functions.

How can we formally define a Turing machine as a function calculator?

Definición

A Turing machine $M = (Q, q_0, q_f, \Sigma, \Gamma, b, \delta)$ *computes a function* $h : \Sigma^* \to \Gamma^*$ *if for every input* $w \in \Sigma^*$ *we have*

$$q_0 w \overset{*}{\vdash} q_f v, \text{ whenever } v = h(w).$$

Turing Machines

What else can an MT be used for?

Since Turing machines have the ability to transform input strings, erase or overwrite symbols, they can also be used as mechanisms to compute functions.

How can we formally define a Turing machine as a function calculator?

Definición

A Turing machine $M = (Q, q_0, q_f, \Sigma, \Gamma, b, \delta)$ *computes a function* $h : \Sigma^* \to \Gamma^*$ *if for every input* $w \in \Sigma^*$ *we have*

$$q_0 w \overset{*}{\vdash} q_f v, \text{ whenever } v = h(w).$$

- *If there exists a Turing machine that computes the function h, then h is said to be Turing-computable.*

Turing Machines

What else can an MT be used for?

Since Turing machines have the ability to transform input strings, erase or overwrite symbols, they can also be used as mechanisms to compute functions.

How can we formally define a Turing machine as a function calculator?

Definición

A Turing machine $M = (Q, q_0, q_f, \Sigma, \Gamma, b, \delta)$ *computes a function* $h : \Sigma^* \to \Gamma^*$ *if for every input* $w \in \Sigma^*$ *we have*

$$q_0 w \overset{*}{\vdash} q_f v, \text{ whenever } v = h(w).$$

- *If there exists a Turing machine that computes the function h, then h is said to be Turing-computable.*
- *The Turing machine model used here is standard, but there are no accepting states.*

Turing Machines

What else can an MT be used for?

Since Turing machines have the ability to transform input strings, erase or overwrite symbols, they can also be used as mechanisms to compute functions.

How can we formally define a Turing machine as a function calculator?

Definición

A Turing machine $M = (Q, q_0, q_f, \Sigma, \Gamma, b, \delta)$ *computes a function* $h : \Sigma^* \to \Gamma^*$ *if for every input* $w \in \Sigma^*$ *we have*

$$q_0 w \overset{*}{\vdash} q_f v, \text{ whenever } v = h(w).$$

- *If there exists a Turing machine that computes the function h, then h is said to be Turing-computable.*
- *The Turing machine model used here is standard, but there are no accepting states.*
- *The state* q_f*, called the final state, is used to terminate the input processing and produce the output.*

Definición

- *Transitions from the final state q_f are not allowed.*

Definición

- *Transitions from the final state q_f are not allowed.*
- *It is worth noting that M must terminate processing the string in configuration $q_f v$, meaning the control unit must be scanning the first symbol of the output v.*

Definición

- *Transitions from the final state q_f are not allowed.*
- *It is worth noting that M must terminate processing the string in configuration $q_f v$, meaning the control unit must be scanning the first symbol of the output v.*
- *If M does not halt on a particular input w, the function h is not defined on w. In other words, the domain of h consists only of those strings for which M halts.*

Definición

- *Transitions from the final state q_f are not allowed.*
- *It is worth noting that M must terminate processing the string in configuration $q_f v$, meaning the control unit must be scanning the first symbol of the output v.*
- *If M does not halt on a particular input w, the function h is not defined on w. In other words, the domain of h consists only of those strings for which M halts.*
- *It is common to write $h(w)! \downarrow$ to indicate that h is defined on w (i.e., h converges on w) and $h(w)! \uparrow$ to indicate that h is not defined on w (i.e., h diverges on w). If the function h is defined for every input w (i.e., if M halts for every w), h is said to be a total function.*

Definición

- *Transitions from the final state q_f are not allowed.*
- *It is worth noting that M must terminate processing the string in configuration $q_f v$, meaning the control unit must be scanning the first symbol of the output v.*
- *If M does not halt on a particular input w, the function h is not defined on w. In other words, the domain of h consists only of those strings for which M halts.*
- *It is common to write $h(w)! \downarrow$ to indicate that h is defined on w (i.e., h converges on w) and $h(w)! \uparrow$ to indicate that h is not defined on w (i.e., h diverges on w). If the function h is defined for every input w (i.e., if M halts for every w), h is said to be a total function.*
- *A function undefined for some inputs is called a partial function.*

Example

Design a Turing machine M that computes the remainder of the division of n by 3, for any natural number $n \geq 1$ written in unary notation (n is written as a sequence of n ones).

Example

Design a Turing machine M that computes the remainder of the division of n by 3, for any natural number $n \geq 1$ written in unary notation (n is written as a sequence of n ones).

Solution

- *The possible remainders of division by 3 are $0, 1$, and 2, so only 3 states, besides q_f, are needed to compute this function.*

Example

Design a Turing machine M that computes the remainder of the division of n by 3, for any natural number $n \geq 1$ written in unary notation (n is written as a sequence of n ones).

Solution

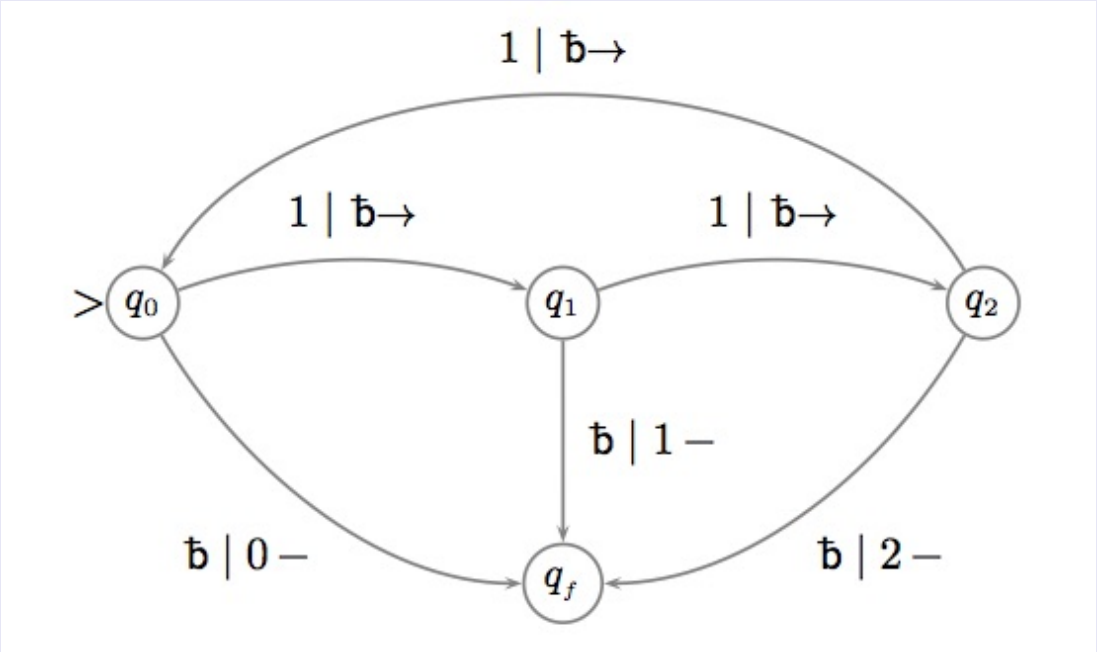
- *The possible remainders of division by 3 are $0, 1$, and 2, so only 3 states, besides q_f, are needed to compute this function.*
- *M scans from left to right the input sequence, erasing the ones and alternatingly transitioning between states q_0 (representing remainder 0), q_1 (remainder 1), and q_2 (remainder 2).*

Solution

- *This is the transition diagram of M :*

Solution

- *This is the transition diagram of M* :



Turing Machines

Can the notion of a Turing-computable function be extended?

Turing Machines

Can the notion of a Turing-computable function be extended?

Yes indeed, the notion of a Turing-computable function can be easily extended to functions of several arguments. Specifically, a function h of k arguments is Turing-computable if for every k-tuple $(w_1, w_2, ..., w_k)$ we have

Can the notion of a Turing-computable function be extended?

Yes indeed, the notion of a Turing-computable function can be easily extended to functions of several arguments. Specifically, a function h of k arguments is Turing-computable if for every k-tuple $(w_1, w_2, ..., w_k)$ we have

$$q_0 w_1 b w_2 b \cdots b w_k b \overset{*}{\vdash} q_f v, \text{ whenever } v = h(w_1, w_2, ..., w_k).$$

Can the notion of a Turing-computable function be extended?

Yes indeed, the notion of a Turing-computable function can be easily extended to functions of several arguments. Specifically, a function h of k arguments is Turing-computable if for every k-tuple $(w_1, w_2, ..., w_k)$ we have

$$q_0 w_1 b w_2 b \cdots b w_k b \overset{*}{\vdash} q_f v, \text{ whenever } v = h(w_1, w_2, ..., w_k).$$

- Note that to write the input on the tape, the k arguments $w_1, w_2, ..., w_k$ are separated by the blank symbol b.

Can the notion of a Turing-computable function be extended?

Yes indeed, the notion of a Turing-computable function can be easily extended to functions of several arguments. Specifically, a function h of k arguments is Turing-computable if for every k-tuple $(w_1, w_2, ..., w_k)$ we have

$$q_0 w_1 b w_2 b \cdots b w_k b \overset{*}{\vdash} q_f v, \text{ whenever } v = h(w_1, w_2, ..., w_k).$$

- Note that to write the input on the tape, the k arguments $w_1, w_2, ..., w_k$ are separated by the blank symbol b.
- Just as before, transitions from the final state q_f are not allowed.

Can the notion of a Turing-computable function be extended?

Yes indeed, the notion of a Turing-computable function can be easily extended to functions of several arguments. Specifically, a function h of k arguments is Turing-computable if for every k-tuple $(w_1, w_2, ..., w_k)$ we have

$$q_0 w_1 b w_2 b \cdots b w_k b \overset{*}{\vdash} q_f v, \text{ whenever } v = h(w_1, w_2, ..., w_k).$$

- Note that to write the input on the tape, the k arguments $w_1, w_2, ..., w_k$ are separated by the blank symbol b.
- Just as before, transitions from the final state q_f are not allowed.
- This definition covers both cases of total functions (defined for all inputs) and partial functions (undefined for some inputs).

Example

Design a Turing machine M that computes the addition function, in unary notation. This function takes two arguments, $h(n, m) = n + m$, where $n, m \geq 1$. With input $1^n b 1^m$, M should produce as output 1^{n+m}. The sequences of ones, 1^n and 1^m, represent the natural numbers n and m, respectively.

Example

Design a Turing machine M that computes the addition function, in unary notation. This function takes two arguments, $h(n, m) = n + m$, where $n, m \geq 1$. With input $1^n b 1^m$, M should produce as output 1^{n+m}. The sequences of ones, 1^n and 1^m, represent the natural numbers n and m, respectively.

Solution

- *M will transform $1^n b 1^m$ into 1^{n+m} by shifting the string 1^m one cell to the left.*

Example

Design a Turing machine M that computes the addition function, in unary notation. This function takes two arguments, $h(n, m) = n + m$, where $n, m \geq 1$. With input $1^n b 1^m$, M should produce as output 1^{n+m}. The sequences of ones, 1^n and 1^m, represent the natural numbers n and m, respectively.

Solution

- *M will transform $1^n b 1^m$ into 1^{n+m} by shifting the string 1^m one cell to the left.*
- *The separator b is overwritten by 1, and the last 1 of 1^m is overwritten by the blank symbol.*

Example

Design a Turing machine M that computes the addition function, in unary notation. This function takes two arguments, $h(n, m) = n + m$, where $n, m \geq 1$. With input $1^n b 1^m$, M should produce as output 1^{n+m}. The sequences of ones, 1^n and 1^m, represent the natural numbers n and m, respectively.

Solution

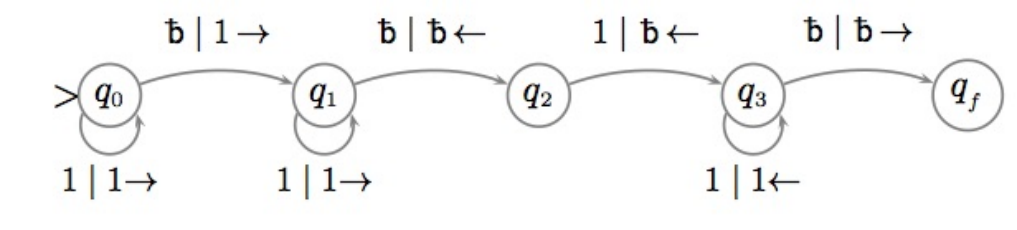
- *M will transform $1^n b 1^m$ into 1^{n+m} by shifting the string 1^m one cell to the left.*
- *The separator b is overwritten by 1, and the last 1 of 1^m is overwritten by the blank symbol.*
- *The transition diagram of the Turing machine implementing this idea is:*

Example

Design a Turing machine M that computes the addition function, in unary notation. This function takes two arguments, $h(n, m) = n + m$, where $n, m \geq 1$. With input $1^n b 1^m$, M should produce as output 1^{n+m}. The sequences of ones, 1^n and 1^m, represent the natural numbers n and m, respectively.

Solution

- *M will transform $1^n b 1^m$ into 1^{n+m} by shifting the string 1^m one cell to the left.*
- *The separator b is overwritten by 1, and the last 1 of 1^m is overwritten by the blank symbol.*
- *The transition diagram of the Turing machine implementing this idea is:*



Agenda del día

1. Grammars and Languages
   - Topic presentation
2. Turing machines
   - Topic presentation
   - Computing functions with Turing machines
   - Exercises